

CAHIER DES CHARGES (CDC)

Projet : SocialSnap – Réseau Social de Partage de Photos

Version 1.0

Rédigé par : Faustin Mardoche,
Jocelyn Youvens Dions,
Eserve Wondy

Établissement : Université ITAC

Professeur : Phawens LOUIS-JEAN

Commanditaire / Contexte : Projet Final N°01 – Développement Mobile (Flutter + Firebase)

Juillet 2026

1. Tableau de gestion des versions

Version	Auteur	Date	Description des modifications
v0.1	Équipe projet	26/06/2026	Création initiale du document
v0.5	Équipe projet	02/07/2026	Ajout des besoins fonctionnels détaillés et de l'architecture technique
v1.0	Équipe projet	03/07/2026	Version complète intégrant l'ensemble des spécifications, cas d'usage, règles de gestion et planning détaillé

2. Sommaire

1. Tableau de gestion des versions.....	2
2. Sommaire.....	2
3. Introduction et contexte général	4
3.1 Contexte.....	4
3.2 Parties prenantes du projet.....	4
3.3 Personas cibles.....	4
3.4 Problématique.....	4
3.5 Objectif général	4
3.6 Objectifs SMART.....	5
3.7 Périmètre du projet.....	5
3.8 Glossaire	5
4. Description de l'existant (État des lieux)	6
4.1 Limites identifiées	6
4.2 Opportunité.....	6
5. Besoins fonctionnels.....	7
5.1 Tableau de traçabilité des besoins fonctionnels.....	7
5.2 Détail des cas d'usage par module	8
5.3 Module Authentification (Priorité : Must Have).....	8
5.3.1 Inscription — BF-001, BF-002.....	8
5.3.2 Connexion et déconnexion — BF-003.....	8
5.3.3 Réinitialisation du mot de passe — BF-004	9
5.4 Module Publication (Priorité : Must Have).....	9
5.4.1 Création d'une publication — BF-005, BF-006.....	9
5.4.2 Modification et suppression d'une publication — BF-007.....	9
5.5 Module Social : Feed, interactions et découverte (Priorité : Must Have / Should Have)	10
5.5.1 Feed personnalisé — BF-008.....	10
5.5.2 Follow / Unfollow — BF-009.....	10
5.5.3 Likes et commentaires en temps réel — BF-010, BF-011.....	11
5.5.4 Notifications in-app — BF-012	11
5.5.5 Découverte et recherche — BF-013, BF-014	11
5.6 Module Profil.....	12
5.6.1 Consultation et édition du profil — BF-015	12

5.7 Priorisation MoSCoW.....	12
6. Besoins non fonctionnels	13
6.1 Tableau de traçabilité des besoins non fonctionnels	13
6.2 Détail par catégorie.....	13
7. Contraintes du projet.....	15
8. Architecture et spécifications techniques.....	16
8.1 Vue d'ensemble.....	16
8.2 Stack technique.....	16
8.3 Modèle de données Firestore	16
Collection users/{uid}	16
Collection posts/{postId}.....	17
Sous-collection posts/{postId}/likes/{uid}	17
Collection follows/{followId}	17
Collection notifications/{notificationId}	17
8.4 Exemple de règles de sécurité Firestore.....	19
8.5 Diagramme de flux : publication d'une photo	19
8.6 Architecture applicative (MVVM + Provider).....	20
8.7 Liste des écrans de l'application.....	20
8.8 Packages Flutter principaux.....	21
9. Exigences de sécurité et conformité	22
9.1 Authentification et autorisation.....	22
9.2 Politique de mots de passe et gestion des sessions	22
9.3 Protection des données.....	22
9.4 Prévention des abus	22
9.5 Sauvegarde et tests de sécurité.....	22
9.6 Consentement et confidentialité.....	22
10. Livrables attendus.....	23
11. Critères de recette et de validation.....	23
11.1 Critères de validation globale	24
11.2 Critères de rejet.....	24
11.3 Gestion des anomalies	24
12. Planning prévisionnel	25
Semaine 1 — Initialisation	25
Semaine 2 — Fonctionnalités principales.....	25
Semaine 3 — Couche sociale	25
Semaine 4 — Polish & Livraison	25
13. Conditions commerciales et juridiques (contexte académique).....	27
Approbation	27

3. Introduction et contexte général

3.1 Contexte

Haïti connaît depuis plusieurs années une pénétration croissante des smartphones et un usage massif des réseaux sociaux comme principal canal d'expression, de partage et de communication au sein de la population, en particulier chez les jeunes urbains. Les plateformes internationales dominantes telles qu'Instagram et Facebook occupent une place centrale dans ces usages, mais elles restent conçues pour un public global : leur interface est majoritairement en anglais, leurs algorithmes de recommandation ne tiennent pas compte des réalités locales (connexions 3G/4G instables, coût élevé de la donnée mobile), et elles n'intègrent aucune spécificité culturelle ou linguistique propre au contexte haïtien, notamment le créole.

Ce constat ouvre une opportunité claire : concevoir une alternative locale, légère et pensée pour ce contexte, capable de proposer une expérience de partage de photos fluide même dans des conditions réseau dégradées, tout en valorisant la langue et la culture locales.

3.2 Parties prenantes du projet

Partie prenante	Rôle	Attentes principales
Équipe étudiante (Faustin Mardoche, Jocelyn Youvens Dions, Eserve Wondy)	Conception, développement, tests et livraison de l'application	Respecter le cahier des charges, produire un livrable fonctionnel et soutenable
Superviseur académique / Jury	Validation des livrables et évaluation du projet	Qualité du code, conformité aux exigences, démonstration convaincante
Utilisateurs finaux (public haïtien)	Utilisation quotidienne de l'application	Application rapide, intuitive, adaptée à leur langue et à leur contexte réseau
Établissement académique (Université ITAC)	Cadre pédagogique du projet	Respect du calendrier et des critères de recette définis

3.3 Personas cibles

Afin de guider les choix fonctionnels et ergonomiques, trois personas représentatifs des utilisateurs visés ont été définis :

Persona	Profil	Objectifs	Freins actuels
Nadège, 24 ans, Port-au-Prince	Étudiante, forte utilisatrice de réseaux sociaux via 4G	Partager des photos de son quotidien avec sa communauté locale	Interfaces en anglais, consommation de données élevée sur Instagram
Jonas, 31 ans, Cap-Haïtien	Petit entrepreneur, met en valeur ses produits en photo	Gagner en visibilité locale sans dépendre d'algorithmes internationaux	Portée limitée sur Facebook, coût des publicités
Sabine, 45 ans, diaspora (Miami)	Reste connectée avec sa famille en Haïti	Suivre les publications de proches en toute simplicité	Multiplicité des applications, complexité des réglages de confidentialité

3.4 Problématique

Comment concevoir et développer une application mobile de partage de photos offrant une expérience sociale complète (publication, interaction, découverte), synchronisée en temps réel et sécurisée, tout en restant performante et économe en données sur les réseaux mobiles haïtiens ?

3.5 Objectif général

Développer SocialSnap, une application mobile cross-platform (Android/iOS) permettant à des utilisateurs authentifiés de partager des photos, de suivre d'autres utilisateurs, de réagir aux publications et de découvrir du contenu pertinent, avec une synchronisation en temps réel assurée par Firebase.

3.6 Objectifs SMART

- Spécifique : livrer une application de réseau social centrée sur le partage de photos, adaptée au marché haïtien.
- Mesurable : authentification, publication, feed personnalisé, interactions sociales et notifications toutes opérationnelles et testées.
- Atteignable : réalisation avec un stack éprouvé (Flutter + Firebase) largement documenté et disposant d'un tier gratuit suffisant pour un MVP.
- Réaliste : périmètre calibré pour un projet final étudiant réalisable en développement individuel ou en petite équipe.
- Temporel : livraison complète en 4 semaines, avec un jalon de validation à la fin de chaque semaine.

3.7 Périmètre du projet

Fonctionnalité	Statut	Justification
Authentification email/mot de passe	Inclus	Socle indispensable à toute interaction sociale
Publication de photos avec légende	Inclus	Fonctionnalité cœur du produit
Feed personnalisé et interactions (like, commentaire, follow)	Inclus	Cœur de l'expérience sociale
Notifications in-app	Inclus	Renforce l'engagement utilisateur
Page de découverte	Inclus	Favorise la croissance organique du réseau
Messagerie privée	Exclu	Complexité et délai incompatibles avec 4 semaines
Stories éphémères	Exclu	Fonctionnalité secondaire, hors MVP
Monétisation / publicité	Exclu	Hors périmètre académique
Version web	Exclu	Le projet cible exclusivement le mobile (Android/iOS)
Modération de contenu par IA	Exclu	Nécessite une infrastructure ML hors budget/temps

3.8 Glossaire

- Feed : fil d'actualité personnalisé présentant les publications des comptes suivis.
- UID : identifiant unique attribué par Firebase Authentication à chaque utilisateur.
- Cloud Functions : fonctions serverless Firebase, utilisées ici pour les compteurs dénormalisés et les notifications.
- MVP : Minimum Viable Product, version minimale mais fonctionnelle du produit.
- MoSCoW : méthode de priorisation Must/Should/Could/Won't have.
- Stream : flux de données Firestore mis à jour en temps réel, consommé via StreamBuilder en Flutter.
- Dénormalisation : duplication contrôlée d'une donnée (ex. compteur de likes) pour optimiser les lectures.

4. Description de l'existant (État des lieux)

Les utilisateurs haïtiens utilisent aujourd'hui principalement Instagram et Facebook pour le partage de photos. Une analyse comparative met en évidence plusieurs limites de ces plateformes vis-à-vis des besoins locaux :

Critère	Instagram	Facebook	TikTok	SocialSnap (cible)
Langue d'interface	Anglais / partiel FR	Anglais / partiel FR	Anglais / partiel FR	Français, créole haïtien priorisé
Optimisation réseaux 3G/4G instables	Faible	Moyenne	Faible	Élevée (compression, pagination)
Pertinence culturelle locale	Faible	Faible	Faible	Élevée (contenu et communauté locale)
Contrôle des données par l'utilisateur	Limité	Limité	Limité	Renforcé (règles Firestore strictes)
Coût de la donnée mobile	Élevé	Moyen	Élevé	Réduit (images compressées 800×800)

4.1 Limites identifiées

- Interface et contenu majoritairement en anglais, peu adaptés à un public francophone et créolophone.
- Algorithmes de recommandation non calibrés pour des connexions 3G/4G instables et un coût de données élevé.
- Absence de fonctionnalités ou de mise en avant spécifiques à la culture et à la communauté haïtiennes.
- Problèmes récurrents de confidentialité et de contrôle limité des données personnelles pour un public local.
- Poids important des médias non compressés, pénalisant les utilisateurs à connexion limitée.

4.2 Opportunité

Aucune solution locale équivalente n'existe à grande échelle sur le marché haïtien. SocialSnap se positionne comme une alternative légère, rapide et culturellement pertinente, en s'appuyant sur une architecture serverless moderne (Firebase) qui permet un développement rapide sans infrastructure dédiée.

5. Besoins fonctionnels

Conformément à la méthodologie recommandée, chaque besoin fonctionnel est d'abord identifié par un code unique (BF-XXX), formulé selon le modèle « Le système DOIT/DEVRAIT/POURRAIT [action] pour [acteur] afin de [objectif] », priorisé selon MoSCoW, puis détaillé sous forme de cas d'usage complet (acteurs, flux nominal, règles de gestion, critères d'acceptation).

5.1 Tableau de traçabilité des besoins fonctionnels

ID	Description du besoin	Priorité	Acteur	Critère d'acceptation
BF-001	Le système DOIT permettre à un visiteur de s'inscrire via email, mot de passe et pseudo unique	Must Have	Visiteur	Compte créé et document Firestore généré en moins de 2 s
BF-002	Le système DOIT envoyer un email de vérification à l'inscription	Must Have	Système	Email reçu en moins de 5 minutes
BF-003	Le système DOIT permettre à un utilisateur inscrit de se connecter et de se déconnecter	Must Have	Utilisateur	Session persistée via authStateChanges
BF-004	Le système DOIT permettre la réinitialisation du mot de passe par email	Must Have	Utilisateur	Lien de réinitialisation reçu et fonctionnel
BF-005	Le système DOIT permettre à un utilisateur authentifié de publier une photo avec légende	Must Have	Utilisateur	Publication visible dans le feed des abonnés en < 2 s après upload
BF-006	Le système DOIT compresser toute image avant upload (800x800 px, qualité 80%)	Must Have	Système	Poids de fichier conforme au seuil défini
BF-007	Le système DOIT permettre à l'auteur de modifier ou supprimer sa publication	Must Have	Auteur	Suppression en cascade des likes/commentaires vérifiée
BF-008	Le système DOIT afficher un feed personnalisé basé sur les abonnements, paginé (12/page)	Must Have	Utilisateur	Chargement incrémental sans doublon via startAfter
BF-009	Le système DOIT permettre de suivre / se désabonner d'un utilisateur	Must Have	Utilisateur	Compteurs followers/following cohérents après opération
BF-010	Le système DOIT permettre de liker une publication, une seule fois par utilisateur	Must Have	Utilisateur	Compteur exact même en cas d'actions simultanées
BF-011	Le système DOIT permettre d'ajouter des commentaires visibles en temps réel	Must Have	Utilisateur	Commentaire visible sur tous les appareils en < 1 s
BF-012	Le système DEVRAIT notifier un utilisateur des likes, commentaires et nouveaux abonnés	Should Have	Système	Badge non-lu exact, aucune auto-notification

ID	Description du besoin	Priorité	Acteur	Critère d'acceptation
BF-013	Le système DEVRAIT proposer une page de découverte des publications populaires/récentes	Should Have	Utilisateur	Grille triée par likesCount ou date, fluide
BF-014	Le système DEVRAIT permettre la recherche d'utilisateurs par pseudo	Should Have	Utilisateur	Résultats pertinents affichés en < 1 s
BF-015	Le système DOIT permettre la consultation et l'édition du profil (photo, bio, nom)	Must Have	Utilisateur	Modifications reflétées immédiatement sur l'application
BF-016	Le système POURRAIT permettre une recherche avancée par hashtags	Could Have	Utilisateur	Résultats filtrés par hashtag exact

5.2 Détail des cas d'usage par module

Chaque besoin identifié ci-dessus est détaillé ci-après, module par module, sous forme de cas d'usage. Les identifiants BF-XXX correspondants sont rappelés dans le titre de chaque cas d'usage pour assurer la traçabilité.

5.3 Module Authentification (Priorité : Must Have)

5.3.1 Inscription — BF-001, BF-002

Acteurs : Visiteur non authentifié

Description : Permettre à un nouvel utilisateur de créer un compte avec un email, un mot de passe et un pseudo unique.

Flux nominal :

1. L'utilisateur accède à l'écran d'inscription depuis l'écran de connexion.
2. Il saisit son email, un mot de passe conforme et un pseudo.
3. L'application vérifie l'unicité du pseudo dans Firestore.
4. Le compte est créé via Firebase Authentication.
5. Un document utilisateur est créé dans la collection users.
6. Un email de vérification est envoyé automatiquement.

Règles de gestion :

- Le pseudo doit être unique sur l'ensemble de la plateforme.
- Le mot de passe doit contenir au minimum 8 caractères.
- L'email doit être vérifié avant l'accès à certaines fonctionnalités sensibles (optionnel selon configuration).

Critères d'acceptation :

- Un utilisateur ne peut pas s'inscrire avec un pseudo déjà utilisé.
- Un message d'erreur explicite est affiché en cas de champ invalide.
- Le document Firestore correspondant est créé dans un délai inférieur à 2 secondes après validation.

5.3.2 Connexion et déconnexion — BF-003

Acteurs : Utilisateur inscrit

Description : Permettre à un utilisateur existant de se connecter et de se déconnecter en toute sécurité.

Flux nominal :

7. L'utilisateur saisit son email et son mot de passe sur l'écran de connexion.
8. Firebase Authentication vérifie les identifiants.
9. En cas de succès, l'utilisateur est redirigé vers le feed principal.

10. La session est persistée via `authStateChanges` pour éviter une reconnexion à chaque ouverture.

Règles de gestion :

- Trois tentatives échouées consécutives déclenchent un message de prudence (sans blocage strict côté client, la limitation réelle étant gérée par Firebase).
- La déconnexion efface la session locale et redirige vers l'écran de connexion.

Critères d'acceptation :

- Un identifiant invalide affiche un message d'erreur clair sans révéler s'il s'agit de l'email ou du mot de passe.
- La session reste active après fermeture et réouverture de l'application tant que l'utilisateur ne s'est pas déconnecté.

5.3.3 Réinitialisation du mot de passe — BF-004

Acteurs : Utilisateur inscrit ayant oublié son mot de passe

Description : Permettre la récupération de l'accès au compte via un email de réinitialisation.

Flux nominal :

11. L'utilisateur clique sur « Mot de passe oublié » depuis l'écran de connexion.
12. Il saisit son adresse email.
13. Firebase Authentication envoie un lien de réinitialisation.
14. L'utilisateur définit un nouveau mot de passe via le lien reçu.

Règles de gestion :

- Le lien de réinitialisation expire après une durée définie par Firebase (par défaut 1 heure).

Critères d'acceptation :

- Un email de réinitialisation est bien reçu pour une adresse existante.
- Aucune information n'est divulguée si l'adresse n'existe pas, afin d'éviter l'énumération de comptes.

5.4 Module Publication (Priorité : Must Have)

5.4.1 Création d'une publication — BF-005, BF-006

Acteurs : Utilisateur authentifié

Description : Permettre à un utilisateur de publier une photo accompagnée d'une légende et de hashtags.

Flux nominal :

15. L'utilisateur capture une photo via l'appareil photo natif ou la sélectionne depuis la galerie.
16. L'image est compressée automatiquement (maximum 800×800 px, qualité 80 %).
17. L'utilisateur ajoute une légende et éventuellement des hashtags.
18. L'application affiche une barre de progression pendant l'upload vers Firebase Storage.
19. Un document est créé dans la collection posts avec la référence de l'auteur et l'horodatage.
20. La publication apparaît dans le feed des abonnés de l'auteur.

Règles de gestion :

- Toute image doit être compressée avant l'upload, sans exception.
- Une publication doit obligatoirement contenir une image ; la légende est optionnelle.
- Les hashtags sont extraits automatiquement des mots précédés du symbole #.

Critères d'acceptation :

- Le poids de l'image uploadée ne dépasse jamais le seuil de compression défini.
- La barre de progression reflète fidèlement l'état de l'upload.
- La publication est visible dans le feed des abonnés en moins de 2 secondes après la fin de l'upload.

5.4.2 Modification et suppression d'une publication — BF-007

Acteurs : Auteur de la publication

Description : Permettre à l'auteur de modifier la légende ou de supprimer définitivement sa publication.

Flux nominal :

21. L'auteur accède à sa publication et sélectionne « Modifier » ou « Supprimer ».
22. En modification, seule la légende peut être éditée.
23. En suppression, l'application demande une confirmation.
24. La suppression entraîne la suppression en cascade des likes et commentaires associés.

Règles de gestion :

- Seul l'auteur d'une publication peut la modifier ou la supprimer (contrôle appliqué également côté règles Firestore).

Critères d'acceptation :

- Un utilisateur autre que l'auteur ne voit pas les options de modification/suppression.
- Après suppression, la publication disparaît immédiatement de tous les feeds concernés.

5.5 Module Social : Feed, interactions et découverte (Priorité : Must Have / Should Have)

5.5.1 Feed personnalisé — BF-008

Acteurs : Utilisateur authentifié

Description : Afficher un fil d'actualité composé des publications des comptes suivis, trié du plus récent au plus ancien.

Flux nominal :

25. L'application interroge Firestore via un Stream filtré sur les comptes suivis par l'utilisateur.
26. Les 12 premières publications sont chargées et affichées.
27. Le défilement déclenche le chargement de la page suivante via startAfter.
28. Un geste de tirer-pour-rafraîchir permet de recharger les publications les plus récentes.

Règles de gestion :

- La pagination est fixée à 12 publications par page.
- Le feed est mis à jour en temps réel dès qu'une nouvelle publication est créée par un compte suivi.

Critères d'acceptation :

- Le feed se charge en moins de 2 secondes sur une connexion 3G simulée.
- Aucun doublon de publication n'apparaît lors de la pagination.

5.5.2 Follow / Unfollow — BF-009

Acteurs : Utilisateur authentifié

Description : Permettre à un utilisateur de suivre ou de se désabonner d'un autre utilisateur.

Flux nominal :

29. L'utilisateur consulte le profil d'un autre utilisateur.
30. Il appuie sur « Suivre » ou « Se désabonner ».
31. Un document est créé ou supprimé dans la collection follows.
32. Les compteurs followersCount et followingCount sont mis à jour de façon dénormalisée.

Règles de gestion :

- Un utilisateur ne peut pas se suivre lui-même.
- La relation de suivi est asymétrique (A peut suivre B sans que B suive A).

Critères d'acceptation :

- Les compteurs affichés restent cohérents après plusieurs opérations successives.
- Le feed de l'utilisateur reflète immédiatement le nouvel abonnement.

5.5.3 Likes et commentaires en temps réel — BF-010, BF-011

Acteurs : Utilisateur authentifié

Description : Permettre aux utilisateurs de réagir aux publications par un like ou un commentaire, avec mise à jour instantanée.

Flux nominal :

- 33.L'utilisateur appuie sur l'icône de like ; un document est ajouté à la sous-collection likes du post.
- 34.Le compteur likesCount du post est incrémenté de façon atomique.
- 35.L'utilisateur peut rédiger un commentaire, stocké avec une référence à l'auteur et à la publication.
- 36.Les autres utilisateurs consultant la publication voient les nouveaux likes/commentaires sans rafraîchissement manuel.

Règles de gestion :

- Un utilisateur ne peut liker une publication qu'une seule fois (clé du document = uid de l'utilisateur).
- La suppression d'un like décrémente le compteur de façon atomique.

Critères d'acceptation :

- Le compteur de likes reste exact même en cas d'actions simultanées de plusieurs utilisateurs.
- Un commentaire apparaît sur tous les appareils connectés en moins d'une seconde après son envoi.

5.5.4 Notifications in-app — BF-012

Acteurs : Utilisateur authentifié

Description : Informer un utilisateur des interactions reçues sur son compte ou ses publications.

Flux nominal :

- 37.Une action (like, commentaire, follow) déclenche la création d'un document dans la collection notifications.
- 38.L'utilisateur destinataire voit un badge non lu sur l'icône de notifications.
- 39.En ouvrant l'écran, la liste des notifications s'affiche, triée par date décroissante.
- 40.Chaque notification consultée est marquée comme lue.

Règles de gestion :

- Une notification n'est jamais générée pour une action de l'utilisateur sur son propre contenu (ex. auto-like).

Critères d'acceptation :

- Le badge de notifications non lues reflète le nombre exact de notifications non consultées.
- Le contenu de la notification renvoie vers la publication ou le profil concerné.

5.5.5 Découverte et recherche — BF-013, BF-014

Acteurs : Utilisateur authentifié

Description : Permettre de découvrir de nouvelles publications populaires ou récentes, et de rechercher des utilisateurs par pseudo.

Flux nominal :

- 41.L'utilisateur accède à l'onglet Découverte.
- 42.L'application affiche une grille de publications triées par popularité (likesCount) ou par date récente.
- 43.L'utilisateur peut basculer vers la recherche et saisir un pseudo partiel.
- 44.Les résultats correspondants s'affichent en temps réel au fur et à mesure de la saisie.

Règles de gestion :

- La recherche s'effectue par préfixe sur le champ username.
- La page de découverte exclut les publications déjà vues dans le feed personnalisé (optionnel selon complexité restante).

Critères d'acceptation :

- Les résultats de recherche apparaissent en moins d'une seconde pour une base de test standard.
- La grille de découverte reste fluide même avec plusieurs dizaines de publications chargées.

5.6 Module Profil

5.6.1 Consultation et édition du profil — BF-015

Acteurs : Utilisateur authentifié

Description : Permettre à l'utilisateur de consulter son profil ou celui d'un tiers, et de modifier ses propres informations.

Flux nominal :

45. L'utilisateur accède à l'écran de profil.
46. Les informations (photo, bio, nom d'affichage, compteurs, grille de publications) sont chargées depuis Firestore.
47. Sur son propre profil, l'utilisateur peut appuyer sur « Modifier » pour éditer sa photo, sa bio et son nom d'affichage.
48. Les modifications sont enregistrées et reflétées immédiatement sur l'ensemble de l'application.

Règles de gestion :

- Seul le propriétaire du profil peut le modifier.
- La photo de profil est compressée avant l'upload, au même titre que les publications.

Critères d'acceptation :

- Le profil affiche des compteurs exacts et à jour.
- Une modification de la photo de profil est visible immédiatement sur les publications existantes de l'utilisateur.

5.7 Priorisation MoSCoW

Priorité	Fonctionnalités concernées
Must (M)	Authentification complète, Publication (création/suppression), Feed personnalisé, Like/Commentaire, Profil
Should (S)	Notifications in-app, Page de découverte, Pagination optimisée
Could (C)	Recherche avancée par hashtags, prise en charge complète du créole dans l'interface
Won't (W)	Messagerie privée, Stories éphémères, Version web

6. Besoins non fonctionnels

Les exigences non fonctionnelles définissent la qualité de service attendue de l'application, indépendamment des fonctionnalités métier. Chacune est identifiée par un code BNF-XXX afin d'assurer sa traçabilité jusqu'aux critères de recette (section 12).

6.1 Tableau de traçabilité des besoins non fonctionnels

ID	Catégorie	Exigence
BNF-001	Performance	Chargement du feed en moins de 2 secondes sur connexion 3G standard
BNF-002	Performance	Publication d'une photo en moins de 5 secondes après compression, hors temps réseau
BNF-003	Sécurité	Règles Firestore limitant lecture/écriture au propriétaire ou aux utilisateurs autorisés
BNF-004	Fiabilité	Message explicite et nouvelle tentative proposée en cas de perte de connexion
BNF-005	Scalabilité	Architecture serverless capable d'absorber une croissance du nombre d'utilisateurs sans refonte
BNF-006	Ergonomie / UX	Respect des principes Material 3 et gestion explicite des états (chargement, vide, erreur)
BNF-007	Optimisation image	Compression systématique à 800×800 px maximum, qualité 80 %
BNF-008	Pagination	12 publications maximum par page, chargement incrémental via startAfter
BNF-009	Internationalisation	Français en priorité, créole haïtien pour les libellés clés de l'interface
BNF-010	Disponibilité	Synchronisation en temps réel via les flux (Stream) Firestore
BNF-011	Maintenabilité	Architecture MVVM, code documenté, séparation models/services/providers/UI
BNF-012	Compatibilité	Android 8.0+ et iOS 13+ minimum
BNF-013	Accessibilité	Contrastes suffisants (WCAG AA visé) et tailles de police adaptables
BNF-014	Sécurité des sessions	Expiration et renouvellement des jetons gérés nativement par Firebase Authentication

6.2 Détail par catégorie

Catégorie	Exigence	Détail / métrique cible
Performance	Temps de chargement du feed	Inférieur à 2 secondes sur une connexion 3G standard
Performance	Temps de publication d'une photo	Inférieur à 5 secondes après compression, hors temps réseau
Sécurité	Contrôle d'accès aux données	Règles Firestore limitant lecture/écriture au propriétaire ou aux abonnés autorisés
Fiabilité	Gestion des erreurs réseau	Message explicite et bouton de nouvelle tentative en cas de perte de connexion
Scalabilité	Croissance du nombre d'utilisateurs	Architecture Firebase serverless capable de monter en charge sans refonte

Catégorie	Exigence	Détail / métrique cible
Utilisabilité (UX)	Cohérence visuelle	Respect des principes Material 3 sur l'ensemble des écrans
Utilisabilité (UX)	Gestion des états	Affichage explicite des états de chargement, vide et erreur sur chaque écran de données
Optimisation image	Poids des fichiers	Compression systématique à 800×800 px maximum, qualité 80 %
Pagination	Volume de données chargées	12 publications maximum par page, chargement incrémental via startAfter
Internationalisation	Langues supportées	Français en priorité, créole haïtien pour les libellés clés de l'interface
Disponibilité	Synchronisation des données	Mise à jour en temps réel via les flux (Stream) Firestore
Maintenabilité	Structure du code	Architecture MVVM avec séparation claire models / services / providers / UI
Compatibilité	Plateformes cibles	Android 8.0+ et iOS 13+ minimum
Accessibilité	Lisibilité	Contrastes suffisants et tailles de police adaptables

7. Contraintes du projet

Le projet est soumis à un ensemble de contraintes techniques, temporelles, humaines et environnementales qui conditionnent les choix effectués tout au long du développement.

Type	Description	Impact sur le projet
Techniques	Utilisation obligatoire de Flutter 3.x et Firebase (Auth, Firestore, Storage)	Orienté tous les choix d'architecture et de packages
Temporelles	Délai global de 4 semaines avec jalons hebdomadaires	Nécessite une priorisation stricte (MoSCoW) et un développement itératif
Ressources humaines	Développement individuel ou en très petite équipe	Limite la parallélisation des tâches, exige une architecture simple à maintenir
Environnementales / réseau	Connexions mobiles 3G/4G instables et coûteuses en Haïti	Impose la compression d'image, la pagination et une gestion robuste du mode hors-ligne partiel
Budgétaires	Aucun budget dédié à des services payants	Utilisation du tier gratuit (Spark) de Firebase dans la mesure du possible
Matérielles	Tests principalement sur émulateur et un nombre limité d'appareils réels	Risque de bugs spécifiques à certains appareils non détectés avant la soutenance
Réglementaires	Absence de cadre légal strict équivalent au RGPD en Haïti	Application volontaire de bonnes pratiques de protection des données par prudence

8. Architecture et spécifications techniques

8.1 Vue d'ensemble

SocialSnap repose sur une architecture client-serveur serverless : l'application Flutter constitue le client unique (Android/iOS), tandis que Firebase assure l'ensemble des services backend (authentification, base de données temps réel, stockage de fichiers). Cette approche permet de concentrer l'effort de développement sur l'expérience utilisateur et la logique métier, sans gestion d'infrastructure serveur dédiée.

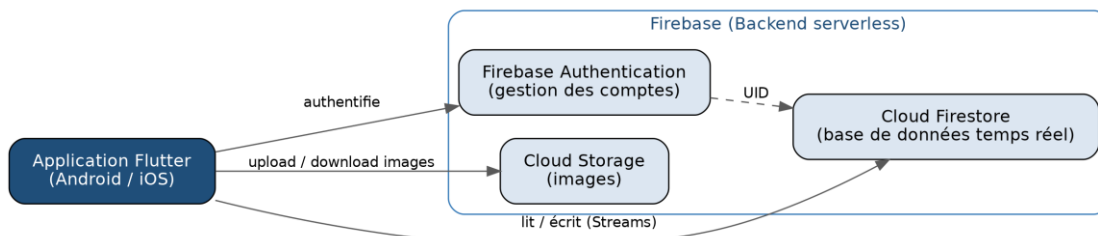


Figure 1 — Diagramme d'architecture système de SocialSnap

8.2 Stack technique

Composant	Choix technique
Frontend	Flutter 3.x (Dart 3, null safety activée)
Navigation	GoRouter, avec redirections basées sur l'état d'authentification
Gestion d'état	Provider + ChangeNotifier
Backend	Firebase (Authentication, Cloud Firestore, Cloud Storage)
Hébergement des médias	Firebase Storage avec règles d'accès dédiées

8.3 Modèle de données Firestore

La base de données est organisée en cinq collections principales. Le schéma ci-dessous illustre les relations logiques entre ces collections, avant le détail champ par champ de chacune d'elles.

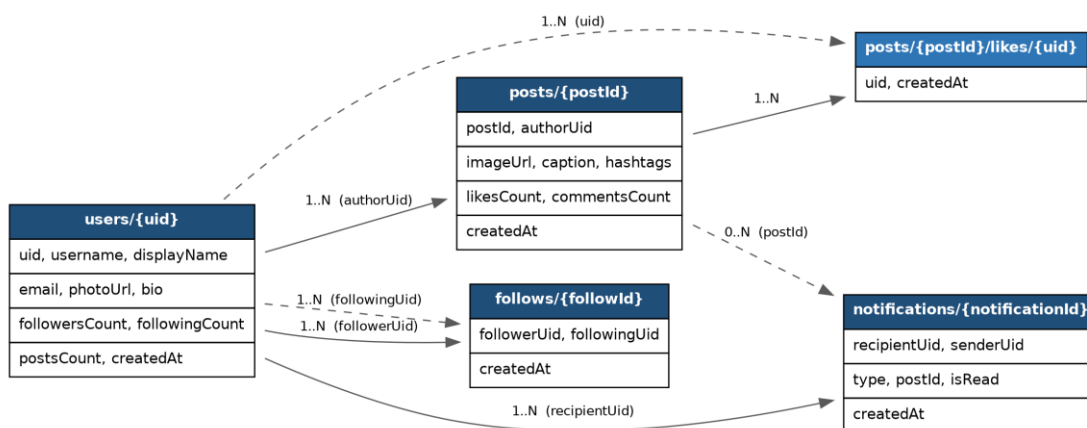


Figure 2 — Schéma des relations entre les collections Firestore

Collection users/{uid}

Champ	Type	Description
uid	string	Identifiant unique Firebase Auth (clé du document)
username	string	Pseudo unique, utilisé pour la recherche

Champ	Type	Description
displayName	string	Nom affiché sur le profil
email	string	Adresse email de connexion
photoUrl	string	URL de la photo de profil (Firebase Storage)
bio	string	Courte description du profil (280 caractères max)
followersCount	number	Compteur dénormalisé du nombre d'abonnés
followingCount	number	Compteur dénormalisé du nombre d'abonnements
postsCount	number	Compteur dénormalisé du nombre de publications
createdAt	timestamp	Date de création du compte

Collection posts/{postId}

Champ	Type	Description
postId	string	Identifiant unique du document (clé)
authorUid	string	Référence à l'auteur (users/{uid})
imageUrl	string	URL de l'image compressée sur Firebase Storage
caption	string	Légende de la publication
hashtags	array<string>	Liste des hashtags extraits de la légende
likesCount	number	Compteur dénormalisé de likes
commentsCount	number	Compteur dénormalisé de commentaires
createdAt	timestamp	Horodatage de publication, utilisé pour le tri et la pagination

Sous-collection posts/{postId}/likes/{uid}

Champ	Type	Description
uid	string	Identifiant de l'utilisateur ayant liké (clé du sous-document)
createdAt	timestamp	Date du like, utile pour les notifications

Collection follows/{followId}

Champ	Type	Description
followId	string	Identifiant composite (ex : {followerUid}_{followingUid})
followerUid	string	Utilisateur qui suit
followingUid	string	Utilisateur suivi
createdAt	timestamp	Date de mise en relation

Collection notifications/{notificationId}

Champ	Type	Description
notificationId	string	Identifiant unique du document

Champ	Type	Description
recipientUid	string	Utilisateur destinataire de la notification
senderUid	string	Utilisateur à l'origine de l'action
type	string	like comment follow
postId	string	Référence à la publication concernée (le cas échéant)
isRead	boolean	Statut de lecture de la notification
createdAt	timestamp	Date de génération de la notification

8.4 Exemple de règles de sécurité Firestore

Les règles ci-dessous illustrent le principe général appliqué : un document n'est modifiable que par son propriétaire, et certaines lectures sont ouvertes aux utilisateurs authentifiés.

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /users/{userId} {
      allow read: if request.auth != null;
      allow write: if request.auth.uid == userId;
    }
    match /posts/{postId} {
      allow read: if request.auth != null;
      allow create: if request.auth.uid == request.resource.data.authorUid;
      allow update, delete: if request.auth.uid == resource.data.authorUid;

      match /likes/{uid} {
        allow read: if request.auth != null;
        allow write: if request.auth.uid == uid;
      }
    }
  }
}
```

8.5 Diagramme de flux : publication d'une photo

Le diagramme suivant illustre, à titre d'exemple représentatif, l'enchaînement des étapes du flux de publication décrit au cas d'usage BF-005/BF-006 (section 5.4.1).

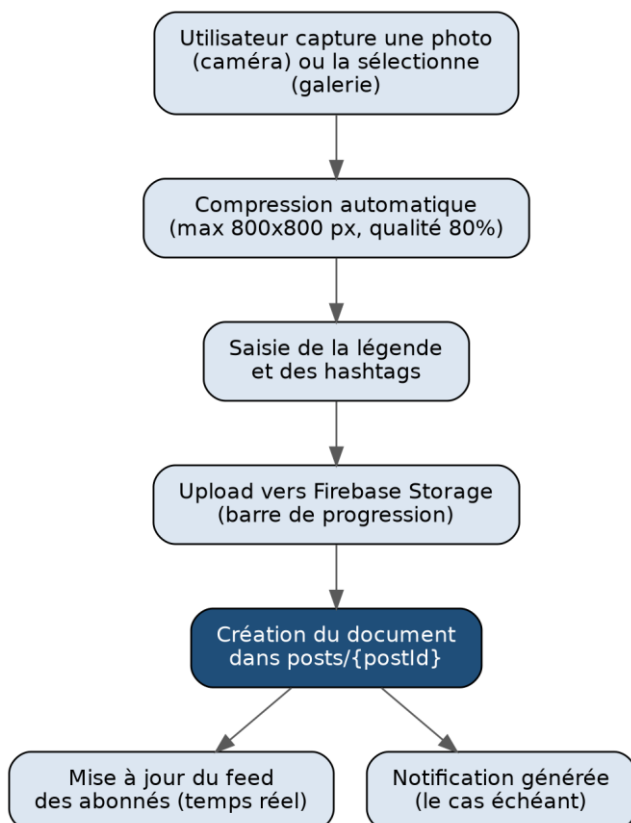


Figure 3 — Diagramme de flux du processus de publication d'une photo

8.6 Architecture applicative (MVVM + Provider)

Le code est structuré selon une architecture MVVM adaptée à Flutter, favorisant la séparation des responsabilités et la testabilité :

- `models/` — classes de données représentant les documents Firestore (User, Post, Comment, Notification).
- `services/` — couche d'accès aux services Firebase (AuthService, FirestoreService, StorageService).
- `providers/` — gestion d'état exposée aux vues via ChangeNotifier (AuthProvider, FeedProvider, ProfileProvider).
- `screens/` — écrans de l'application, un dossier par fonctionnalité majeure.
- `widgets/` — composants d'interface réutilisables (carte de publication, barre de progression, avatar).

8.7 Liste des écrans de l'application

Écran	Objectif	Composants clés
Splash / Auth Gate	Vérifier l'état de connexion au démarrage	StreamBuilder sur authStateChanges
Connexion	Authentifier un utilisateur existant	Champs email/mot de passe, lien mot de passe oublié
Inscription	Créer un nouveau compte	Champs email, mot de passe, pseudo unique
Feed principal	Afficher les publications des comptes suivis	Liste paginée, pull-to-refresh, cartes de publication
Découverte	Mettre en avant les publications populaires/récentes	Grille de vignettes, tri par popularité
Détail publication	Afficher une publication, ses likes et commentaires	Zone de commentaires, bouton like animé
Publication (création)	Capturer ou sélectionner puis publier une photo	Caméra, galerie, éditeur de légende, barre de progression
Profil (soi/autre)	Afficher les informations et publications d'un utilisateur	Grille de publications, compteurs, bouton follow/unfollow
Édition du profil	Modifier photo, bio et nom d'affichage	Formulaire avec aperçu de la photo
Recherche	Rechercher des utilisateurs par pseudo	Champ de recherche avec suggestions en temps réel
Notifications	Lister les notifications reçues	Liste avec indicateur lu/non lu
Paramètres	Gérer le compte et la déconnexion	Options de sécurité, déconnexion, suppression de compte

8.8 Packages Flutter principaux

Package	Version cible	Usage
firebase_core	^3.x	Initialisation de Firebase dans l'application
firebase_auth	^5.x	Gestion de l'authentification (email/mot de passe)
cloud_firestore	^5.x	Base de données temps réel (feed, likes, commentaires, follows)
firebase_storage	^12.x	Stockage des images de publication et de profil
provider	^6.x	Gestion d'état de l'application (ChangeNotifier)
go_router	^14.x	Navigation déclarative et gestion des routes protégées
image_picker	^1.x	Sélection d'images depuis la galerie
camera	^0.11.x	Capture de photo via l'appareil photo natif
flutter_image_compress	^2.x	Compression des images avant upload
cached_network_image	^3.x	Mise en cache des images du feed pour limiter la consommation de données
intl	^0.19.x	Formatage des dates et prise en charge de la localisation

9. Exigences de sécurité et conformité

9.1 Authentification et autorisation

- Authentification gérée exclusivement par Firebase Authentication (email/mot de passe).
- Vérification systématique de l'identité de l'utilisateur avant toute opération d'écriture sensible.
- Règles Firestore appliquant le principe du moindre privilège : chaque utilisateur ne peut modifier que ses propres données.

9.2 Politique de mots de passe et gestion des sessions

- Mot de passe exigé de 8 caractères minimum, conformément à la politique par défaut de Firebase Authentication.
- Jetons de session (ID tokens) gérés nativement par Firebase, avec expiration automatique après 1 heure et renouvellement transparent via le refresh token tant que l'utilisateur reste actif.
- Déconnexion explicite invalidant la session locale ; possibilité d'ajouter une révocation forcée des jetons côté Firebase Admin en cas de compromission suspectée.
- Verrouillage progressif des tentatives de connexion successives géré par les mécanismes anti-abus natifs de Firebase Authentication.

9.3 Protection des données

- Toutes les communications entre l'application et Firebase transitent en HTTPS/TLS, garantissant le chiffrement en transit.
- Les données au repos bénéficient du chiffrement natif appliqué par Google Cloud sur Firestore et Cloud Storage.
- Les images sont stockées dans des chemins Firebase Storage propres à chaque utilisateur, avec règles d'accès dédiées.
- Les données sensibles (email) ne sont jamais exposées publiquement dans les documents accessibles en lecture par d'autres utilisateurs.

9.4 Prévention des abus

- Limitation logique du nombre de requêtes via la pagination (12 éléments par page) afin de réduire les risques d'abus et de surcoût.
- Validation côté client et côté règles Firestore des champs critiques (ex. authorUid lors de la création d'une publication).
- Journalisation implicite des actions critiques via les horodatages createdAt présents sur chaque document.

9.5 Sauvegarde et tests de sécurité

- Le code source est versionné sur GitHub, ce qui évite toute perte en cas de problème sur le poste de développement.
- Les règles Firestore sont vérifiées avec le simulateur Firebase (Firestore Emulator Suite) avant chaque démonstration.
- Les scénarios de sécurité de base sont testés manuellement : tentative de lecture/écriture sur les données d'un autre utilisateur, accès sans authentification.

9.6 Consentement et confidentialité

- Présentation d'une politique de confidentialité simplifiée lors de l'inscription, informant l'utilisateur des données collectées.
- Possibilité pour l'utilisateur de supprimer son compte et les données associées depuis les paramètres.

10. Livrables attendus

Livrable	Détail / Format
Code source	Dépôt GitHub structuré, historique de commits clair, README détaillé (installation, configuration Firebase, lancement)
Application fonctionnelle	Fichier APK signé pour Android ; build iOS si un environnement macOS est disponible
Documentation technique	Description de l'architecture, du modèle de données Firestore et des règles de sécurité
Manuel utilisateur	Guide illustré des principales fonctionnalités, destiné à un utilisateur non technique
Rapport de tests	Comptes rendus des scénarios de test exécutés, captures d'écran à l'appui
Support de soutenance	Présentation synthétique du projet, de la démarche et des résultats obtenus

11. Critères de recette et de validation

La recette est conduite par l'étudiant lui-même puis jouée devant le superviseur/jury, selon les scénarios de test ci-dessous couvrant l'ensemble des fonctionnalités Must Have (traçables aux ID BF-XXX de la section 5) et les principales fonctionnalités Should Have.

#	Scénario	Étapes clés	Résultat attendu
T01	Inscription réussie	Saisir email valide, mot de passe conforme et pseudo unique, valider	Compte créé, email de vérification envoyé, redirection vers le feed
T02	Inscription avec pseudo déjà pris	Saisir un pseudo existant	Message d'erreur explicite, formulaire non soumis
T03	Connexion échouée	Saisir un mot de passe incorrect	Message d'erreur, aucun accès accordé
T04	Réinitialisation de mot de passe	Demander la réinitialisation via l'email	Email de réinitialisation reçu, mot de passe modifiable
T05	Publication d'une photo	Capturer/sélectionner une photo, ajouter une légende, publier	Photo compressée, uploadée, visible dans le feed des abonnés
T06	Suppression d'une publication	Supprimer une publication en tant qu'auteur	Publication supprimée du feed et des données associées (likes, commentaires)
T07	Like en temps réel	Liker une publication depuis deux comptes distincts	Compteur mis à jour instantanément sur les deux appareils
T08	Ajout d'un commentaire	Rédiger et envoyer un commentaire	Commentaire visible immédiatement, compteur incrémenté
T09	Follow / Unfollow	Suivre puis se désabonner d'un utilisateur	Feed mis à jour en conséquence, compteurs ajustés
T10	Réception d'une notification	Un utilisateur A like une publication de B	B reçoit une notification in-app en temps réel

#	Scénario	Étapes clés	Résultat attendu
T11	Pagination du feed	Faire défiler le feed au-delà de 12 publications	Chargement automatique de la page suivante via startAfter
T12	Comportement hors-ligne	Couper la connexion pendant une action	Message d'erreur clair et possibilité de réessayer
T13	Recherche d'utilisateur	Rechercher un pseudo partiel	Suggestions pertinentes affichées en temps réel

11.1 Critères de validation globale

La validation finale du projet est prononcée lorsque l'ensemble des critères suivants est satisfait :

- 100 % des fonctionnalités classées Must Have sont opérationnelles et testées avec succès.
- Les scénarios de test ci-dessus sont exécutés sur émulateur et sur au moins un appareil Android réel.
- Aucune anomalie bloquante n'est constatée lors de la démonstration finale.
- Le code est versionné, commenté et organisé selon l'architecture définie en section 8.
- Le superviseur ou le jury valide formellement la conformité du livrable au présent cahier des charges.

11.2 Critères de rejet

Un livrable est refusé lors de la recette si l'une des conditions suivantes est constatée :

- Une fonctionnalité Must Have est absente, non fonctionnelle ou produit un résultat différent du critère d'acceptation défini en section 5.
- Une règle de sécurité Firestore permet à un utilisateur d'accéder ou de modifier des données qui ne lui appartiennent pas.
- L'application se ferme de manière inattendue (crash) lors de l'exécution d'un scénario de test standard.
- Le code source n'est pas accessible, n'est pas versionné, ou ne correspond pas à l'application démontrée.

11.3 Gestion des anomalies

Toute anomalie détectée pendant les tests est consignée (description, étapes de reproduction, gravité), corrigée par l'étudiant, puis revérifiée avant la démonstration finale.

12. Planning prévisionnel

Le planning est organisé en quatre semaines, chacune correspondant à une phase du développement. Le détail jour par jour est fourni ci-dessous afin de faciliter le suivi d'avancement.

Semaine 1 — Initialisation

Jour	Tâches	Livrable / Jalon
J1-J2	Initialisation du dépôt GitHub, configuration Flutter et Firebase (Auth, Firestore, Storage)	Projet initialisé et connecté à Firebase
J3-J4	Implémentation de l'authentification (inscription, connexion, réinitialisation)	Flux d'authentification fonctionnel
J5	Mise en place de GoRouter et de la persistance de session (authStateChanges)	Navigation protégée opérationnelle
J6-J7	Tests unitaires et correctifs, revue de code de la semaine	Auth OK + Repo structuré

Semaine 2 — Fonctionnalités principales

Jour	Tâches	Livrable / Jalon
J8-J9	Modélisation des collections Firestore (users, posts) et services associés	Modèles de données validés
J10-J11	Capture/sélection photo, compression et upload vers Firebase Storage	Upload d'image fonctionnel avec barre de progression
J12-J13	Création, modification et suppression de publications	CRUD publication complet
J14	Tests et corrections	Publication fonctionnelle de bout en bout

Semaine 3 — Couche sociale

Jour	Tâches	Livrable / Jalon
J15-J16	Feed personnalisé basé sur les abonnements (Stream Firestore) et pagination	Feed temps réel opérationnel
J17	Système Follow / Unfollow avec compteurs dénormalisés	Relations sociales fonctionnelles
J18-J19	Likes en temps réel et fil de commentaires	Interactions sociales complètes
J20-J21	Écran de profil (affichage et édition)	Profil utilisateur finalisé

Semaine 4 — Polish & Livraison

Jour	Tâches	Livrable / Jalon
J22-J23	Notifications in-app (like, commentaire, follow)	Notifications fonctionnelles
J24	Page de découverte et recherche d'utilisateurs	Découverte et recherche opérationnelles

Jour	Tâches	Livrable / Jalon
J25-J26	Tests complets, corrections de bugs, optimisation des performances	Application stabilisée
J27	Génération de l'APK, rédaction de la documentation et du manuel utilisateur	Livrables finalisés
J28	Répétition et préparation de la soutenance	Soutenance prête

13. Conditions commerciales et juridiques (contexte académique)

- Le code source reste la propriété de l'étudiant, avec un droit d'utilisation accordé à l'institution académique à des fins d'évaluation.
- L'ensemble des packages open-source utilisés doit respecter leurs licences respectives (MIT, BSD ou équivalent), vérifiées avant intégration.
- Les données utilisées à des fins de test sont fictives ou anonymisées et ne font l'objet d'aucune diffusion externe.
- Aucune responsabilité commerciale n'est engagée dans le cadre de ce projet académique, celui-ci n'ayant pas vocation à être exploité commercialement en l'état.

Approbation

Rédacteur : Faustin Mardoche Date : _____

Rédacteur : Jocelyn Youvens Dions Date : _____

Rédacteur : Eserve Wondy Date : _____

Valideur /Profeseur Phawens LOUIS-JEAN : _____ Date : _____